AWS DATABASE SERVICES

Amazon DynamoDB Complete Study Notes

A comprehensive reference covering fundamentals, design patterns, capacity modes, streams, global tables, best practices, and exam review questions.

DATE	SERVICE	PLATFORM	SECTIONS
April 12, 2026	Amazon DynamoDB	AWS NoSQL	10 Topics

Table of Contents

01 Introduction

02 DynamoDB Fundamentals

03 Primary Keys

04 Secondary Indexes (GSI & LSI)

05 Capacity Modes

06 Consistency Models

07 DynamoDB Streams

08 Global Tables

09 DynamoDB Accelerator (DAX)

10 Production-Level Knowledge

11 Tips & Best Practices

12 Pitfalls & Remedies

13 Summary & Key Takeaways

14 Review Questions

01 Introduction

Amazon DynamoDB is AWS's flagship NoSQL database service, designed to deliver **single-digit millisecond performance at any scale**. Unlike traditional relational databases that struggle with massive scale, DynamoDB provides consistent performance whether you're storing gigabytes or petabytes, handling hundreds or millions of requests per second.

This serverless database automatically scales, replicates across multiple Availability Zones, and provides built-in security, backup, and global replication — all without managing servers, clusters, or

infrastructure.

> **Core Philosophy:** Instead of normalizing data and performing complex joins at query time like SQL databases, DynamoDB requires you to *design your data model around your access patterns*. Understanding partition keys, sort keys, secondary indexes, and query patterns is essential — poor key design leads to hot partitions, throttling, and failed queries.

DynamoDB Streams enable event-driven architectures, triggering Lambda functions on every data change. Global Tables replicate data across multiple regions with sub-second latency. DynamoDB Accelerator (DAX) provides microsecond caching. These capabilities make DynamoDB ideal for gaming leaderboards, IoT telemetry, mobile backends, e-commerce carts, and any application requiring extreme scale.

02 DynamoDB Fundamentals

What is DynamoDB?

DynamoDB is a fully managed NoSQL database service providing:

- ✓ **Fast Performance** — Single-digit millisecond latency at any scale
- ✓ **Scalability** — Unlimited storage, automatic scaling
- ✓ **High Availability** — Replication across 3 AZs, 99.99% SLA
- ✓ **Serverless** — No servers to manage, pay per request or throughput
- ✓ **Flexible Schema** — Store varying attributes per item
- ✓ **Global Replication** — Multi-region active-active replication

Core Concepts

```
Table          Collection of items (like a SQL table, but without fixed schema)
  Item           Individual record (like a SQL row)
    Attributes   Key-value pairs (like SQL columns)
  Primary Key    Uniquely identifies each item
    Partition Key  Required — determines data distribution
    Sort Key       Optional — enables range queries
  Secondary Indexes  Alternative access patterns
```

DynamoDB vs Relational Databases

Feature	DynamoDB	SQL Databases
Data Model	Key-value, document	Relational tables
Schema	Flexible, per item	Fixed schema
Queries	By key or index	SQL with joins
Joins	**Not supported (denormalize)**	**Supported**
Scaling	**Horizontal, automatic**	Vertical, manual
Consistency	Eventual or strong	Strong (ACID)
Performance	**Consistent, ms latency**	Variable with scale
Use Case	Web scale, gaming, IoT	Complex queries, transactions

03 Primary Keys

The primary key is the **most critical design decision** in DynamoDB.

Type 1 — Partition Key Only (Simple Primary Key)

The partition key alone uniquely identifies each item.

userId (PK)	name	email
user123	John Doe	john@example.com
user456	Jane Doe	jane@example.com

✓ Get user by `userId` — O(1) lookup

✗ Get users by name — Full table scan (slow)

Type 2 — Partition Key + Sort Key (Composite Primary Key)

Enables range queries and one-to-many relationships.

customerId (PK)	orderDate (SK)	total

customer123	2025-01-15	$299.99
customer123	2025-01-20	$149.99
customer456	2025-01-18	$499.99

✓ Get all orders for customer — Query by `customerId`

✓ Get orders in date range — Query `customerId` with `orderDate BETWEEN`

✓ Get specific order — Get by `customerId + orderDate`

✗ Get all orders by date (all customers) — Requires index

Partition Key Design Principles

GOOD PARTITION KEY

- High cardinality (many unique values)
- Even access distribution
- Predictable query pattern

Examples: `userId`, `deviceId`, `sessionId`

BAD PARTITION KEY

- Low cardinality (few unique values)
- Uneven access distribution
- Unpredictable access

Examples: `status` (3–4 values), `date` (sequential)

Partition Key Calculation

How DynamoDB determines physical partition:

```
Partition = hash(partition_key) mod number_of_partitions
```

Example with 4 partitions:
hash("user123") mod 4 = **Partition 2**
hash("user456") mod 4 = **Partition 1**
hash("user789") mod 4 = **Partition 2**

Even distribution is critical for performance.

04 Secondary Indexes (GSI & LSI)

Secondary indexes provide alternative access patterns beyond the primary key.

Global Secondary Index (GSI)

- Different partition key and/or sort key from the base table
- Spans all table partitions
- Has its own provisioned capacity (separate from table)
- Eventually consistent reads only
- Can be created or deleted at any time
- Sparse — only contains items with index key attributes

> **GSI Example — Game Leaderboard:**
> Base Table PK: `userId` | SK: `gameId`
> GSI PK: `gameId` | SK: `score`
> Use case: Get top scores for a game sorted by score DESC

Local Secondary Index (LSI)

- Same partition key as table, different sort key
- Shares table partitions and capacity
- Supports **strong consistency**
- Must be created at table creation — cannot add later
- Not sparse — contains all items from the partition

> **LSI Example — Orders by Status:**
> Base Table PK: `customerId` | SK: `orderDate`
> LSI PK: `customerId` | SK: `status`
> Use case: Get all pending orders for a customer

GSI vs LSI Comparison

Feature	GSI	LSI
Partition Key	Can be different	Must be same as table
Sort Key	Can be different	Different from table
Capacity	Separate	Shared with table

Consistency	**Eventual only**	**Eventual or strong**
Creation Time	**Anytime**	**Table creation only**
Limits per table	20	5
Sparse	**Yes (only items with keys)**	No (all items)

05 Capacity Modes

1. Provisioned Capacity

Specify read/write capacity units (RCU/WCU) in advance.

Unit	Definition
1 RCU	1 strongly consistent read/sec for items up to 4 KB *— OR — 2 eventually consistent reads/sec for items up to 4 KB*
1 WCU	1 write/sec for items up to 1 KB

Example pricing (us-east-1): RCU: $0.00013/hr | WCU: $0.00065/hr
100 RCU + 50 WCU = **~$11.25/month**

Best for: Predictable traffic, steady load, cost optimization priority.

2. On-Demand Capacity

Pay per request — no capacity planning needed.

Pricing: Reads: $0.25 per million | Writes: $1.25 per million
Example: 500K reads + 500K writes = **$0.75/month**

Best for: Unpredictable traffic, spiky workloads, new applications (learning phase), infrequent access.

Decision Matrix

CHOOSE PROVISIONED WHEN

- Traffic patterns are predictable
- Steady, consistent baseline load
- Cost optimization is priority
- Can forecast capacity needs

CHOOSE ON-DEMAND WHEN

- Traffic patterns are unknown
- Highly variable/spiky traffic
- New application (learning phase)
- Infrequent access patterns

06 Consistency Models

EVENTUALLY CONSISTENT (DEFAULT)

- Read might not reflect recent write (< 1 second delay)
- Higher throughput (2x RCU)
- Lower latency
- Lower cost

Use case: Non-critical reads, high throughput scenarios

Example: Profile picture update — brief delay is acceptable

STRONGLY CONSISTENT

- Read reflects all successful writes
- Half the throughput (1x RCU)
- Slightly higher latency
- Same cost per consistent read

Use case: Critical reads requiring latest data

Example: Bank balance after a transfer — must be accurate

07 DynamoDB Streams

Streams capture a **time-ordered sequence of item-level modifications**, enabling event-driven architectures.

Stream View Types

#	View Type	What is Captured

1	`KEYS_ONLY`	Only partition and sort keys
2	`NEW_IMAGE`	Entire item after modification
3	`OLD_IMAGE`	Entire item before modification
4	`NEW_AND_OLD_IMAGES`	Both before and after states

Stream Record Structure

```
{
  "eventName": "INSERT | MODIFY | REMOVE",
  "eventTime": "2025-01-15T10:30:00Z",
  "keys": { "userId": "user123" },
  "newImage": { "userId": "user123", "status": "active" },
  "oldImage": { "userId": "user123", "status": "pending" }
}
```

Use Cases

- ✓ **Aggregation** — Maintain counters and summaries
- ✓ **Replication** — Sync to Elasticsearch, S3, or other systems
- ✓ **Notifications** — Trigger alerts on data changes
- ✓ **Audit Logging** — Log all item modifications
- ✓ **Cross-region replication** — Global Tables use streams internally
- ✓ **Event-driven architectures** — Trigger AWS Lambda on every change

> **Retention:** Stream records are retained for **24 hours**.

08 Global Tables

Global Tables provide **multi-region, active-active replication** with sub-second latency.

Architecture

```
Region 1 (us-east-1)
  DynamoDB Table (read/write)
  Stream (replication source)
         |
  [sub-second replication]
         |
Region 2 (eu-west-1)
  DynamoDB Table (read/write)
  Stream (replication source)
         |
Region 3 (ap-southeast-1)
  DynamoDB Table (read/write)
```

Features

- ✓ **Active-active** — Write to any region
- ✓ **Sub-second replication** — Typically < 1 second lag
- ✓ **Conflict resolution** — Last-writer-wins (by timestamp)
- ✓ **Automatic failover** — If a region fails, use another

Conflict Resolution

Scenario: Concurrent writes to same item in different regions

Region 1 (10:00:00.100): Update item A, status = "active"
Region 2 (10:00:00.200): Update item A, status = "inactive"
Result: Region 2 wins (last writer wins based on timestamp)
Final state in ALL regions: status = "inactive"

Best practice: Design to minimize conflicts by using immutable operations (append instead of update) or partitioning data by region when possible.

09 DynamoDB Accelerator (DAX)

DAX is an **in-memory cache for DynamoDB** providing microsecond response times — a 10x+ improvement over standard DynamoDB reads.

Architecture

```
Application
    DAX Cluster (in-memory cache)
      Cache HIT  → Return cached data (microseconds)
      Cache MISS → Query DynamoDB → Cache result → Return
    DynamoDB Table
```

Performance Comparison

Operation	Without DAX	With DAX	Improvement
GetItem	1–5 ms	**0.1–0.3 ms**	10x+
Query	5–20 ms	**0.5–2 ms**	10x+

When to Use DAX

USE DAX WHEN

- Read-heavy workloads (90%+ reads)
- Microsecond latency required
- Repeated reads of same items
- High read throughput needed

SKIP DAX WHEN

- Write-heavy workloads
- Millisecond latency is acceptable
- Most reads are unique queries
- Cost-sensitive (DAX adds cost)

10 Production-Level Knowledge

Time-Series Data Patterns

PATTERN 1 — TIME-BASED PARTITION KEYS

BAD DESIGN (HOT PARTITION)

```
"sensorId": "sensor123",  # PK
"timestamp": "...",       # SK
# All data → same partition!
```

GOOD DESIGN (DISTRIBUTED)

```
"sensorId_month":
"sensor123#2025-01",
"timestamp": "...",
# Distributes by month
```

PATTERN 2 — WRITE SHARDING

Add a random suffix (shard number) to the partition key to spread writes across multiple physical partitions.

PATTERN 3 — TTL FOR AUTOMATIC EXPIRATION

Enable TTL on a table to automatically delete expired items (within 48 hours of expiry). TTL deletions do not consume write capacity.

PATTERN 4 — HOT PARTITION MITIGATION

Use Contributor Insights in CloudWatch to identify hot keys, then redesign or shard accordingly.

Advanced Query Patterns

- **Composite Sort Key Pattern** — Use hierarchical sort keys like `type#id#timestamp` to enable multiple access patterns from a single index
- **Sparse Index Pattern** — Create a GSI on an attribute that only some items have; the index only contains those items, making it smaller and cheaper
- **One-to-Many Pattern** — Use composite primary key (PK = parent, SK = child) to store related items in the same partition

Backup and Restore

- **Point-in-Time Recovery (PITR)** — Continuous backups for the last **35 days**; restore to any second within that window
- **On-demand backups** — Full table backups retained until explicitly deleted; no performance impact

Monitoring & Alerting

- Alert on **ReadThrottleEvents** / **WriteThrottleEvents**
- Alert on high **ConsumedReadCapacityUnits** vs. provisioned
- Alert on **SystemErrors** (DynamoDB internal errors)

- Alert on **SuccessfulRequestLatency** exceeding thresholds
- Use **CloudWatch Contributor Insights** to identify hot keys

11 Tips & Best Practices

Key Design Tips

TIP 1

Design for Access Patterns First

List all access patterns before designing keys. Design keys to support patterns. Denormalize as needed.

TIP 2

Use Composite Sort Keys

Hierarchical sort keys like `type#id#timestamp` enable multiple query patterns from a single table.

TIP 3

Avoid Hot Partitions

Use high cardinality partition keys. Ensure access is evenly distributed across all partition keys.

TIP 4

Use GSI Projections

Include only needed attributes in GSI projections to reduce index size, cost, and read capacity consumption.

TIP 5

Batch Operations

Use `BatchGetItem` and `BatchWriteItem` to reduce API calls and improve throughput efficiency.

TIP 6

Filter Expressions Carefully

Filters apply AFTER reading items — you're still charged for all scanned items. Design keys/indexes to avoid filtering.

Capacity Planning Tips

TIP 7 — START WITH ON-DEMAND

1. Start with on-demand billing for new applications
2. Monitor usage for 30 days
3. Analyze patterns
4. Switch to provisioned if cost-effective (typically saves 30%+)

TIP 8 — ALWAYS ENABLE AUTO SCALING

For provisioned capacity, always configure auto-scaling to handle unexpected traffic spikes without manual intervention.

Monitoring Tips

TIP 9 — MONITOR THROTTLING

Throttling indicates: provisioned capacity too low, hot partition issue, or burst capacity depleted. Set CloudWatch alarms immediately.

TIP 10 — USE CLOUDWATCH CONTRIBUTOR INSIGHTS

Shows most accessed partition keys, most throttled keys, and request distribution — essential for identifying hot key skew before it impacts production.

12 Pitfalls & Remedies

! Pitfall 1 — Hot Partition Problem

Uneven data distribution or access patterns causing throttling on specific partitions despite having available capacity.

WHY IT HAPPENS

- Low cardinality partition key (e.g. `status` with only 3 values)
- Sequential keys (e.g. timestamp as partition key)
- Celebrity/popular item problem — one item accessed heavily

IMPACT

- Throttling despite available provisioned capacity
- Poor performance for affected requests
- Wasted capacity on idle partitions

REMEDY

1. Identify hot partitions with CloudWatch Contributor Insights
2. Redesign partition key to use higher cardinality
3. Add a GSI with better key distribution
4. Apply write sharding — add random suffix to partition key

PREVENTION

✓ Design partition keys with high cardinality

✓ Use write sharding for time-series data

✓ Test at scale before production

✓ Use composite keys to distribute load

! Pitfall 2 — Inefficient Scan Operations

Using `Scan` operations on large tables causing slow performance and high costs — reads and charges for every item in the table.

WHY IT HAPPENS

- Not understanding the difference between `Query` and `Scan`
- Missing appropriate indexes
- Legacy SQL mindset ("SELECT * WHERE ...")

IMPACT

- Slow queries (seconds to minutes on large tables)
- High cost — charged for all items scanned, not returned
- Consumed capacity affects all other operations

REMEDY

1. Replace `Scan` with `Query` using `KeyConditionExpression`
2. Add a GSI or LSI to support the required access pattern
3. If `Scan` is unavoidable, use `Parallel Scan` with segments

PREVENTION

- ✓ Design indexes to support all query patterns upfront
- ✓ Avoid `Scan` operations in production code
- ✓ Test with production-scale data before launch
- ✓ Monitor `ScanCount` metric in CloudWatch

! Pitfall 3 — Exceeding Item Size Limit

Items exceeding the **400 KB size limit** cause write failures at runtime.

WHY IT HAPPENS

- Storing large documents or files directly in items
- Denormalizing too much data into one item
- Attribute count growing over time

> **Item Size Calculation:** Sum of all attribute name lengths + attribute value sizes + binary data sizes
> Max item size: **400 KB**

REMEDY

1. Store large blobs in S3 — store only the S3 reference in DynamoDB
2. Compress large text data before storing
3. Split large items into multiple items with a common PK

PREVENTION

- ✓ Monitor item sizes during development
- ✓ Store large objects in S3, not DynamoDB
- ✓ Use GSI projections to reduce index sizes
- ✓ Design data model with size limits in mind from the start

13 Summary & Key Takeaways

Amazon DynamoDB provides serverless NoSQL database capabilities with single-digit millisecond performance at any scale. Understanding key design, secondary indexes, capacity modes, streams, and global tables is essential for building high-performance applications.

1 **Design keys for access patterns**

List all patterns first, then design keys and indexes to support them efficiently. Never design schema first.

2 **Avoid hot partitions**

Use high cardinality partition keys, shard time-series data, and monitor with Contributor Insights continuously.

3 **Choose the right capacity mode**

On-demand for variable/unknown traffic; provisioned with auto-scaling for predictable, steady loads.

4 **Use indexes wisely**

GSI for different keys (eventual consistency only); LSI for same partition key (supports strong consistency, must be created at table creation).

5 **Leverage Streams**

Enable event-driven architectures, aggregations, cross-system replication, and audit trails without polling.

6 **Monitor throttling proactively**

Set up CloudWatch alarms for throttle events, analyze access patterns, and optimize before production traffic peaks.

7 **Optimize costs continuously**

Right-size capacity, use projections, clean up unused indexes, consider switching billing modes as traffic patterns mature.

Bottom line: Understanding DynamoDB deeply enables you to build applications that scale from zero to millions of requests per second while maintaining consistent performance and controlling costs.

14 Review Questions

Q1. Maximum DynamoDB item size?

a) 64 KB b) **400 KB** c) 4 MB d) No limit

Answer: B — Maximum item size is 400 KB.

Q2. Which index type supports strong consistency reads?

a) GSI reads b) **LSI reads** c) Eventual consistent reads d) Cross-region reads

Answer: B — LSI supports strong consistency; GSI only supports eventual consistency.

Q3. A DynamoDB RCU (read capacity unit) provides?

a) 1 strongly consistent read/sec up to 4 KB b) 2 eventually consistent reads/sec up to 4 KB c) **Both A and B** d) Neither

Answer: C — 1 RCU = 1 strong consistent OR 2 eventual consistent reads/sec (4 KB).

Q4. Maximum number of GSIs per table?

a) 5 b) 10 c) **20** d) Unlimited

Answer: C — Maximum 20 GSIs per table.

Q5. When must a Local Secondary Index (LSI) be created?

a) Anytime b) **At table creation only** c) Before adding data d) After table creation

Answer: B — LSI must be created when the table is created; it cannot be added later.

Q6. DynamoDB Stream retention period?

a) 6 hours b) **24 hours** c) 7 days d) 30 days

Answer: B — Stream records are retained for 24 hours.

Q7. Global Tables replication is?

a) Synchronous b) **Asynchronous** c) Manual d) Not supported

Answer: B — Global Tables use asynchronous replication (typically sub-second).

Q8. DAX provides what latency improvement?

a) 2x faster b) 5x faster c) **10x+ faster (ms to µs)** d) No improvement

Answer: C — DAX reduces latency from milliseconds to microseconds (10x+ improvement).

Q9. On-demand billing charges per?

a) Hour b) Request c) GB stored d) **Both B and C (request + GB stored)**

Answer: D — On-demand charges per request AND per GB stored.

Q10. Point-in-time recovery (PITR) retains backups for?

a) 7 days b) **35 days** c) 90 days d) 1 year

Answer: B — PITR retains backups for 35 days.

Q11. Which causes a hot partition?

a) High cardinality partition key b) **Low cardinality partition key** c) Many GSIs d) Large items

Answer: B — Low cardinality (few unique values) causes hot partitions.

Q12. Write capacity unit (WCU) supports?

a) **1 write/sec up to 1 KB** b) 1 write/sec up to 4 KB c) 2 writes/sec up to 1 KB d) 1 write/sec up to 10 KB

Answer: A — 1 WCU = 1 write/sec for items up to 1 KB.

Q13. What does a Scan operation do?

a) Reads specific items b) **Reads all items in table** c) Reads items matching key d) Reads from index only

Answer: B — Scan reads the entire table (all items), then optionally filters.

Q14. TTL deletions occur within?

a) Immediately b) Within minutes c) **Within 48 hours** d) Manual only

Answer: C — TTL deletes items within 48 hours of their expiration time.

Q15. Best partition key characteristic?

a) Low cardinality b) Sequential values c) **High cardinality with even access** d) Rarely accessed

Answer: C — High cardinality with even access distribution is ideal.